FA1 DevOps Activity - Quick Notes

Project: Automated Deployment of Campus Lost & Found Portal

Goal: Automate server creation, configuration and application deployment using Terraform, Ansible and Docker.

1. Tool Purpose

Tool	Purpose
Terraform	Creates AWS EC2 infrastructure.
Ansible	Configures the server and installs Docker.
Docker	Packages and runs the web application.
GitHub	Stores project files and version history.

2. Simple Workflow

Laptop -> Terraform -> AWS EC2 -> Ansible -> Docker -> Lost & Found App -> Browser

3. Easy Step-by-Step

Step 1 - Create project folders

Create terraform/, ansible/, app/ and README.md.

Step 2 - Create web page

Create app/index.html with a simple Campus Lost & Found page.

Step 3 - Dockerize

Create app/Dockerfile:

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Step 4 - Create AWS key pair

In AWS EC2, create key pair fa1-key and save fa1-key.pem. Do not upload it to GitHub.

Step 5 - Write Terraform files

Use main.tf to create an Ubuntu EC2 instance + security group. Keep port 22 for SSH and port 80 for the website.

Step 6 - Configure AWS

Run aws configure and enter your AWS credentials.

Step 7 - Run Terraform

From the terraform folder:

```
terraform init
terraform validate
terraform plan
terraform apply
```

Copy the EC2 public IP after creation.

Step 8 - Create Ansible inventory

Add the EC2 IP to ansible/inventory.ini:

```
[web]
YOUR_IP ansible_user=ubuntu ansible_ssh_private_key_file=../fa1-key.pem
```

FA1 DevOps Activity - Quick Notes

Continue from Step 9

Step 9 - Test Ansible

```
ansible all -i inventory.ini -m ping
```

Expected result: ping: pong.

Step 10 - Run Ansible playbook

The playbook should: update packages -> install Docker -> copy app files -> build Docker image -> start container.

```
ansible-playbook -i inventory.ini deploy.yml
```

Step 11 - Open the application

Open http://YOUR_EC2_PUBLIC_IP in the browser. The Campus Lost & Found Portal should appear.

Step 12 - Push to GitHub

Use git init, git add ., git commit, then push to GitHub. Add *.pem and Terraform state files to .gitignore.

Step 13 - Final cleanup

After faculty verification, remove AWS resources:

```
terraform destroy
```

4. What to Show Faculty

1. Problem statement and workflow diagram
2. Terraform files and successful terraform apply
3. AWS EC2 instance running
4. Ansible ping and playbook result
5. Dockerfile
6. Working website in browser
7. GitHub repository

5. Viva - Very Short Answers

Question	Answer
DevOps	Combines development and operations with automation.
Terraform	Creates infrastructure.
Ansible	Configures infrastructure.
Docker	Runs the application in a container.
IaC	Infrastructure managed using code.
Image vs Container	Image = blueprint; Container = running instance.

6. One-Line Result

The project successfully automated AWS infrastructure provisioning with Terraform, server configuration with Ansible, and application deployment using Docker.

> Remember: Terraform = CREATE | Ansible = CONFIGURE | Docker = RUN